

模型推理链路与性能指标

1. Tensor：承载数据

Tensor 是一个多维数组，但它不只有数值，还包含描述这些数值的信息。

主要属性：

- `shape`：各维度大小。
- `dtype`：每个元素的数据类型。
- `device`：数据存放的位置。
- `stride/layout`：数据在内存中的排列方式。

Tensor 可以承载：

- 用户输入
- 图片像素
- Token ID
- 中间计算结果
- 模型输出
- 梯度
- 模型权重

2. Parameter：模型学到的权重

Parameter 本质上也是 Tensor，但它被明确标记为模型的可学习参数。

例如：

```
layer = torch.nn.Linear(128, 64)

print(layer.weight.shape) # [64, 128]
print(layer.bias.shape)  # [64]
```

其中：

- `weight` 是 Parameter。
- `bias` 是 Parameter。
- 它们会被优化器发现和更新。
- 它们会进入模型的 `state_dict`。

训练时：

```
Parameter
  ↓ 参与前向计算
产生 Loss
  ↓ backward
产生 Gradient
  ↓ optimizer.step
Parameter 被更新
```

推理时不再训练，所以 Parameter 通常：

- 不计算梯度；
- 不被修改；
- 作为只读权重参与计算。

Parameter 与普通 Tensor 的主要区别不是存储结构，而是它在模型系统中的身份：

```
x = torch.randn(10)                # 普通 Tensor
w = torch.nn.Parameter(torch.randn(10)) # 可学习参数
```

Parameter 解决的问题是：

哪些 Tensor 是模型需要学习、保存和加载的权重？

3. Operator：逻辑计算

Operator 是从数学或计算图角度定义的操作。

常见 Operator：

- MatMul：矩阵乘法
- Add：加法
- Softmax：概率归一化
- RMSNorm：归一化
- Attention：注意力计算
- Convolution：卷积
- ReLU：激活函数

例如：

```
y = torch.matmul(x, weight)
```

从逻辑上看，这里执行了一个 MatMul Operator。

Operator 通常只描述：

输入是什么
输出是什么
数学语义是什么

它不一定规定具体怎么使用 GPU 线程、Shared Memory 或 SIMD 指令。

例如，MatMul 的逻辑是：

$$C = A \times B$$

但它可能由不同实现完成：

- CPU 普通循环
- CPU SIMD
- cuBLAS
- Tensor Core
- 自定义 CUDA Kernel
- 融合 Kernel

Operator 解决的问题是：

从数学和计算图角度，需要执行什么计算？

4. Kernel：硬件上的具体实现

Kernel 是 Operator 在某种硬件上的底层实现。

例如，同一个 MatMul Operator 可以对应：

CPU MatMul Kernel
CUDA FP32 Kernel
CUDA FP16 Tensor Core Kernel
INT8 Tensor Core Kernel
Apple Metal Kernel

CUDA Kernel 示例：

```
__global__ void add_kernel(  
    const float* a,
```

```

const float* b,
float* output,
int size) {

    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < size) {
        output[i] = a[i] + b[i];
    }
}

```

这个 Kernel 明确包含：

- GPU 线程如何编号；
- 每个线程处理哪个元素；
- 如何读取和写入内存；
- 使用多少线程和线程块。

Operator 与 Kernel 不一定一一对应：

一个 Operator 对应多个 Kernel

MatMul 可以根据以下条件选择不同 Kernel：

- GPU 型号
- dtype
- shape
- Batch Size
- 内存布局
- 是否使用 Tensor Core

多个 Operator 融合成一个 Kernel

例如：

```
MatMul → Add Bias → ReLU
```

原本可能启动三个 Kernel，经过算子融合后可能只启动一个：

```
Fused MatMul + Bias + ReLU Kernel
```

这样可以减少：

- Kernel Launch 开销
- 中间 Tensor
- 显存读写
- GPU 同步

Kernel 解决的问题是：

这项计算具体如何在当前 CPU/GPU 上高效执行？

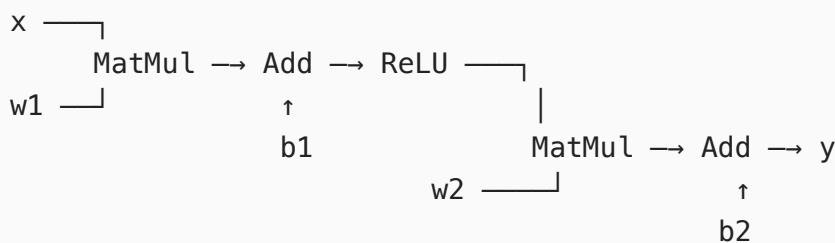
5. Computational Graph：描述计算依赖

计算图由 Operator 和 Tensor 之间的数据依赖组成。

假设：

```
h = x @ w1 + b1
a = torch.relu(h)
y = a @ w2 + b2
```

对应的计算图：



计算图告诉系统：

- 哪些 Operator 要执行；
- 每个 Operator 的输入和输出；
- Operator 之间的依赖关系；
- 哪些计算可以并行；
- 哪些中间 Tensor 可以释放或复用；
- 哪些 Operator 可以融合。

在训练时，框架还会根据前向计算构建反向传播所需的依赖关系。

计算图解决的问题是：

整个模型包含哪些计算，它们必须按照什么依赖顺序执行？

6. Framework：开发和训练模型的工具

Framework 是开发者用来构造、训练、调试和导出模型的完整工具体系。

典型例子：

- PyTorch
- TensorFlow
- JAX

PyTorch 提供：

- Tensor API
- Autograd
- `nn.Module`
- Optimizer
- Dataset/DataLoader
- CPU/CUDA 支持
- 分布式训练
- 模型保存和导出
- Profiler

例如：

```
class Model(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = torch.nn.Linear(128, 64)

    def forward(self, x):
        return torch.relu(self.linear(x))
```

Framework 特别适合：

- 编写模型；
- 训练模型；
- 自动求导；
- 快速实验；
- 调试算法；
- 导出模型。

Framework 解决的问题是：

开发者如何方便地定义、训练、调试和导出模型？

7. Runtime：执行模型的引擎

Runtime 负责加载模型，并在指定硬件上高效执行。

典型例子：

- ONNX Runtime
- TensorRT
- OpenVINO
- PyTorch Runtime

Runtime 常见工作：

1. 加载模型或计算图。
2. 检查输入 shape 和 dtype。
3. 优化计算图。
4. 融合 Operator。
5. 规划 Tensor 内存。
6. 选择 CPU 或 GPU 执行后端。
7. 为 Operator 选择合适的 Kernel。
8. 调度 Kernel 执行。
9. 返回输出 Tensor。

例如，一个 ONNX 模型可能包含：

```
MatMul → Add → ReLU
```

Runtime 可能将它优化为：

```
FusedLinearReLU
```

然后根据环境选择：

```
CPU Execution Provider  
CUDA Execution Provider  
TensorRT Execution Provider
```

Runtime 一般不负责：

- HTTP/gRPC 请求；
- 用户认证；
- 请求队列；
- 超时和重试；
- 服务监控；
- 多用户流量治理。

Runtime 解决的问题是：

已经得到一个模型后，如何在指定硬件上正确、高效地执行？

8. Serving：把模型变成在线服务

Serving 位于 Runtime 外面，负责处理在线请求。

常见组件：

HTTP/gRPC Server
请求解析
身份验证
有界队列
Batcher
Scheduler
Runtime
后处理
超时
监控
限流

完整链路例如：

客户端发送 HTTP 请求
↓
Serving 解析 JSON
↓
请求进入有界队列
↓
Batcher 合并多个请求
↓
Runtime 执行模型
↓
Runtime 调用 GPU Kernel
↓
Serving 拆分 Batch 结果



序列化并返回响应

Serving 需要关心：

- QPS 和并发量；
- 排队时间；
- 动态批处理；
- 超时和取消；
- 背压与限流；
- 模型版本；
- 健康检查；
- p50、p95、p99 延迟；
- 日志、指标和 Tracing。

例如，Runtime 的模型计算只需要 20ms，但用户看到的延迟可能是 200ms：

网络传输	10ms
请求解析	5ms
队列等待	130ms
Batch 等待	20ms
Runtime 计算	20ms
后处理和序列化	15ms

总延迟	200ms

这说明模型执行没有变慢，问题出在 Serving 的队列或流量管理。

Serving 解决的问题是：

如何让多个真实用户稳定、高效、可观测地调用模型？

Framework、Runtime、Serving 的区别

层级	主要使用者	核心职责	例子
Framework	模型开发者	构造、训练、调试和导出模型	PyTorch
Runtime	推理工程师	加载、优化并执行模型	ONNX Runtime、TensorRT
Serving	后端/Infra 工程师	网络、队列、Batch、超时和监控	Triton、自研 C++ 服务

一个典型流程是：

```
PyTorch Framework
  ↓ 训练并导出
ONNX 模型
  ↓ 加载
TensorRT / ONNX Runtime
  ↓ 被服务封装
Triton 或自研 C++ Serving
  ↓
客户端通过 HTTP/gRPC 调用
```

一句话记忆

Tensor：数据放在哪里、长什么样。
Parameter：哪些数据是模型学到的权重。
Operator：数学上要计算什么。
Kernel：硬件上具体怎么计算。
Graph：这些计算如何连接和依赖。
Framework：如何开发和训练模型。
Runtime：如何优化并执行模型。
Serving：如何让真实用户稳定调用模型。

性能指标：

服务能力：Latency、QPS、Throughput、Concurrency、Batch Size
延迟拆分：Queue Time、Batch Wait、Compute Time、Serialization Time
延迟分布：p50、p95、p99
LLM 指标：TTFT、TPOT、ITL、tokens/s

1. Latency：单次请求延迟

Latency 表示一个请求从开始到完成经历的时间。

例如：

```
客户端发出请求：10:00:00.000
客户端收到结果：10:00:00.120

Latency = 120ms
```

服务端内部可以拆成：

```
总延迟
= 网络时间
+ 序列化/反序列化时间
+ Queue Time
+ Batch Wait
+ Compute Time
+ 后处理时间
```

Latency 回答：

一个用户需要等多久？

需要明确测量边界：

- 客户端端到端延迟；
- 服务端请求延迟；
- 纯模型执行延迟。

这三者不是同一个指标。

2. QPS：每秒完成的请求数

QPS 是 Queries Per Second，即系统每秒处理多少个请求。

10 秒内完成 500 个请求

$$\text{QPS} = 500 \div 10 = 50$$

QPS 回答：

系统一秒能服务多少个请求？

注意必须说明“请求”的含义。例如 LLM 请求可能分别生成 10 个和 1,000 个 Token，因此只报告 QPS 不能完整表示工作量。

到达 QPS 与完成 QPS

还需要区分：

- Arrival QPS：每秒到达多少请求。
- Completion QPS：每秒完成多少请求。

如果：

到达速率 = 30 QPS
处理上限 = 25 QPS

每秒约有 5 个请求无法及时处理，队列会不断增长，最终导致高延迟、超时或拒绝请求。

3. Throughput：吞吐量

Throughput 表示单位时间内完成的总工作量。

在不同场景中，单位不同：

图像推理：images/s
普通服务：requests/s
训练：samples/s
LLM 推理：tokens/s
网络传输：MB/s 或 GB/s

QPS 是一种以“请求”为单位的吞吐量，但 Throughput 的含义更广。

例如：

服务 A：10 QPS，每个请求输出 1,000 Tokens
服务 B：20 QPS，每个请求输出 100 Tokens

输出吞吐量：

$A = 10 \times 1,000 = 10,000 \text{ tokens/s}$
 $B = 20 \times 100 = 2,000 \text{ tokens/s}$

虽然 B 的 QPS 更高，但 A 实际处理的 Token 更多。

Throughput 回答：

系统单位时间完成了多少实际工作？

4. Concurrency：并发数

Concurrency 表示同一时间正在系统中处理或等待的请求数量。

这些请求可能处于：

- 网络接收；
- 请求解析；

- 队列等待；
- 等待组成 Batch；
- GPU 执行；
- 结果返回。

例如系统中同时存在 32 个未完成请求：

```
Concurrency = 32
```

根据 Little's Law，在系统稳定的前提下：

```
Concurrency ≈ QPS × 平均延迟（秒）
```

例如：

```
QPS = 100  
平均延迟 = 0.2 秒
```

```
Concurrency ≈ 100 × 0.2 = 20
```

并发不等于吞吐

提高并发最初可能提高 GPU 利用率和吞吐量，但超过系统容量后：

```
并发继续提高  
↓  
Queue Time 增长  
↓  
p95/p99 恶化  
↓  
超时增加  
↓  
有效吞吐可能下降
```

Concurrency 回答：

| 同一时刻有多少请求在系统中？

5. Batch Size：单次计算处理的样本数量

Batch Size 表示一次模型执行中一起处理多少个样本或请求。

例如：

```
Batch Size = 1
```

每次只处理一个样本。

```
Batch Size = 16
```

一次 GPU 执行处理 16 个样本。

较大的 Batch 通常可以：

- 提高 GPU 利用率；
- 分摊 Kernel Launch 开销；
- 提高总体吞吐量。

但也可能：

- 增加单请求等待时间；
- 增加显存占用；
- 增加单次 Compute Time；
- 恶化 p95/p99。

静态和动态 Batch

静态 Batch：

```
固定每次处理 8 个请求
```

动态 Batch：

```
最多收集 8 个请求  
最多等待 2ms  
满足任一条件就执行
```

Batch Size 回答：

一次模型执行合并处理多少个样本？

延迟拆分

假设一次服务端请求耗时：

Queue Time	20ms
Batch Wait	3ms
Compute Time	15ms
Serialization Time	2ms
其他时间	5ms

服务端延迟	45ms

6. Queue Time：队列等待时间

Queue Time 表示请求进入服务端队列后，到被 Worker、Scheduler 或 Batchers 接收之前的时间。

```
进入队列：10:00:00.000
离开队列：10:00:00.030

Queue Time = 30ms
```

Queue Time 升高通常意味着：

- 到达速度超过处理能力；
- Worker 或 GPU 不足；
- 前面的请求太慢；
- Batch Size 或调度策略不合理；
- 出现资源争用；
- 队列没有合理的背压机制。

如果 Compute Time 稳定，但端到端延迟和 Queue Time 上升，问题通常在服务容量或调度层，而不是模型本身。

7. Batch Wait：等待组成 Batch 的时间

Batch Wait 表示请求被 Batchers 接收后，为了与其他请求组成更大的 Batch 而等待的时间。

例如：

```
请求到达 Batchers：00.000
Batch 发往 GPU：    00.004

Batch Wait = 4ms
```

动态批处理需要在两件事之间权衡：

- 多等一会儿
- 更容易组成大 Batch
- 吞吐量可能提高
- 单请求延迟增加

因此通常同时配置：

最大 Batch Size = 8
最大等待时间 = 2ms

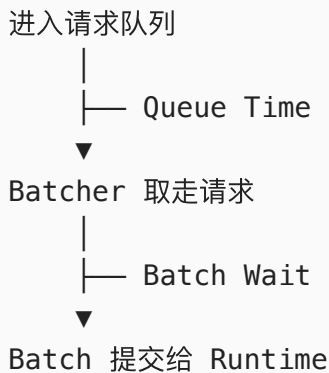
只要达到 Batch Size 或最大等待时间，都会立即执行。

Batch Wait 回答：

为了凑 Batch，这个请求额外等了多久？

Queue Time 与 Batch Wait 的边界

不同系统的定义可能不同。一种常见划分是：



使用指标前必须确认具体系统如何定义。

8. Compute Time：计算时间

Compute Time 表示模型真正执行计算所花费的时间。

可能包括：

- Runtime 调度；
- CPU Operator；
- CUDA Kernel；
- GPU 运算；
- CPU/GPU 数据传输；

- 后处理计算。

但不同系统的统计边界可能不同。例如：

- 有的只统计 GPU Kernel 时间；
- 有的统计整个 Runtime 调用；
- 有的包含 Host-to-Device 数据传输。

分析报告中应明确口径。

例如：

```
Queue Time 从 10ms 升至 200ms  
Compute Time 一直约 20ms
```

说明模型计算没有明显变慢，瓶颈更可能是请求量超过系统容量。

如果 Compute Time 本身升高，应检查：

- Batch Size 是否变大；
- 输入 shape 或序列长度是否增加；
- GPU 是否被其他进程占用；
- 是否发生降频；
- 是否选择了较慢的 Kernel；
- 是否存在额外的数据复制；
- 显存不足是否触发特殊处理。

Compute Time 回答：

| 模型或硬件真正花了多少时间完成计算？

9. Serialization Time：序列化时间

Serialization Time 表示数据在程序对象与传输格式之间转换所用的时间。

请求进入时：

```
JSON / Protobuf / 字节流  
    ↓ 反序列化  
Tensor 或程序对象
```

返回结果时：

Tensor 或程序对象
↓ 序列化
JSON / Protobuf / 字节流

常见成本来源：

- 大型 JSON 数组；
- Base64 图片；
- Protobuf 编解码；
- Tensor 转 Python List；
- GPU Tensor 复制回 CPU；
- 数据类型转换；
- 多次内存复制。

严格来说，通常应该分别记录：

- Deserialization Time；
- Serialization Time。

如果模型只计算 2ms，但把巨大 Tensor 转为 JSON 使用了 30ms，那么优化 CUDA Kernel 的价值很小。

Serialization Time 回答：

输入输出格式转换花了多久？

延迟百分位

10. p50：中位延迟

p50 表示 50% 的请求延迟不超过这个值。

例如把 100 个请求延迟从小到大排列：

p50 = 30ms

意味着：

- 约 50 个请求不超过 30ms；
- 约 50 个请求超过 30ms。

p50 代表典型用户体验。

11. p95：长尾延迟

p95 表示 95% 的请求延迟不超过这个值，最慢的约 5% 请求超过它。

```
p95 = 80ms
```

表示约 20 个请求中有 1 个超过 80ms。

p95 常用于：

- 服务性能目标；
- 用户体验监控；
- 容量规划；
- 性能回归判断。

2. p99：极端长尾延迟

p99 表示 99% 的请求延迟不超过这个值，最慢的约 1% 请求超过它。

```
p99 = 300ms
```

如果服务每天处理 100 万次请求：

```
1% × 1,000,000 = 10,000
```

仍有约 1 万次请求比 300ms 更慢，所以 1% 并不一定是少数用户。

为什么不能只看平均值？

假设延迟是：

```
10, 11, 11, 12, 12, 13, 13, 14, 15, 200ms
```

其中大部分请求在 10–15ms，但有一个请求为 200ms。

```
平均值 = 31.1ms
```

```
中位数 = 12.5ms
```

平均值既不能准确代表正常请求，也不能清楚表达最慢请求。

因此性能报告通常至少包含：

mean、p50、p95、p99、max

百分位的直观关系

p50：普通用户通常等多久

p95：较慢用户等多久

p99：极端长尾有多严重

百分位需要足够多的样本。只有几十个请求时，p99 的统计意义很弱。

LLM 专用指标

传统分类模型通常一次计算就返回结果，而 LLM 会逐个生成 Token：

```
请求到达
  ↓ 等待、Prefill
生成第一个 Token
  ↓ Decode
Token 2
  ↓ Decode
Token 3
  ↓
生成结束
```

因此只看总延迟不足以描述体验。

13. TTFT：首 Token 延迟

TTFT 是 Time To First Token，即从请求发出到收到第一个输出 Token 的时间。

```
发送请求：00.000
收到首 Token：00.180
```

```
TTFT = 180ms
```

TTFT 通常包含：

- 网络和请求解析；
- Queue Time；
- Batch Wait；
- Tokenization；
- Prompt Prefill；

- 第一次 Decode;
- 首 Token 返回。

影响 TTFT 的重要因素:

- Prompt 长度;
- 排队时间;
- Prefill 计算量;
- 动态批处理等待;
- GPU 负载;
- 调度策略。

TTFT 回答:

用户多久能看到模型开始回答?

聊天产品通常非常关注 TTFT, 因为快速出现第一个字会降低用户感受到的等待。

14. TPOT: 平均每个输出 Token 的时间

TPOT 是 Time Per Output Token, 通常表示首 Token 之后, 生成每个后续 Token 平均需要多少时间。

一种常见计算方式:

$$\begin{aligned} \text{TPOT} \\ &= (\text{请求总时间} - \text{TTFT}) \div (\text{输出 Token 数} - 1) \end{aligned}$$

例如:

总延迟 = 2,160ms
TTFT = 180ms
输出 = 100 Tokens

$$\begin{aligned} \text{TPOT} &= (2160 - 180) \div 99 \\ &= 20\text{ms/token} \end{aligned}$$

TPOT 越低, 生成速度越快。

近似换算:

$$\text{单请求生成速度} \approx 1000 \div \text{TPOT}(\text{ms})$$

如果：

```
TPOT = 20ms/token
```

则单请求大约：

```
1000 ÷ 20 = 50 tokens/s
```

TPOT 回答：

回答开始后，模型持续输出的速度有多快？

15. ITL：相邻 Token 的时间间隔

ITL 是 Inter-Token Latency，即两个相邻输出 Token 到达用户之间的时间。

例如 Token 到达时间：

```
Token 1: 180ms  
Token 2: 198ms  
Token 3: 220ms  
Token 4: 260ms
```

对应 ITL：

```
18ms、22ms、40ms
```

TPOT 通常是整个请求的平均生成速度；ITL 可以表示每个 Token 的具体到达间隔及其分布：

```
平均 ITL  
p50 ITL  
p95 ITL  
p99 ITL
```

即使平均 TPOT 很好，偶尔出现很大的 ITL，用户仍会感觉输出卡顿。

ITL 回答：

连续输出过程中是否均匀，有没有突然停顿？

16. tokens/s：Token 吞吐量

tokens/s 表示系统每秒处理或生成多少 Token。

但必须说明统计口径。

单请求输出速度

单个请求生成 100 Tokens, 用时 2 秒

输出速度 = 50 tokens/s

它与 TPOT 近似互为倒数：

$\text{tokens/s} \approx 1000 \div \text{TPOT(ms)}$

系统总输出吞吐

10 个并发请求

每个请求约 40 tokens/s

系统输出吞吐可能接近 400 tokens/s

实际结果还会受到调度、Batch 和显存带宽影响。

输入与输出 Token 吞吐

应该区分：

- Input tokens/s：Prefill 处理 Prompt 的速度。
- Output tokens/s：Decode 生成 Token 的速度。
- Total tokens/s：输入与输出合计。

因为 Prefill 和 Decode 的计算特征不同：

- Prefill 更容易进行大规模并行计算；
- Decode 每一步通常只生成一个新 Token，更容易受显存带宽和调度影响。

tokens/s 回答：

单个请求或整个系统每秒处理了多少 Token？

指标之间的关系

Latency、QPS 与 Concurrency

系统稳定时，可以用 Little's Law 近似：

$$\text{Concurrency} \approx \text{QPS} \times \text{平均 Latency (秒)}$$

例如：

$$\text{QPS} = 50$$

$$\text{平均 Latency} = 0.2\text{s}$$

$$\text{Concurrency} \approx 10$$

Batch Size、吞吐和延迟

通常：

Batch Size 增大

↓

GPU 利用率可能提高

↓

Throughput 增加

但同时：

Batch Size 增大

↓

Batch Wait 和 Compute Time 可能增加

↓

Latency、p95、p99 可能恶化

因此不存在永远最好的 Batch Size，需要在延迟目标下寻找最大吞吐。

LLM 总延迟估算

如果输出包含 N 个 Token：

$$\text{总延迟} \approx \text{TTFT} + (N - 1) \times \text{TPOT}$$

例如：

$$\text{TTFT} = 180\text{ms}$$

$$\text{输出 Token 数} = 100$$

TPOT = 20ms

总延迟 $\approx 180 + 99 \times 20$
= 2160ms
= 2.16s

一句话记忆

Latency: 一次请求要等多久。

QPS: 每秒完成多少请求。

Throughput: 每秒完成多少实际工作。

Concurrency: 同时有多少请求在系统中。

Batch Size: 一次计算合并多少样本。

Queue Time: 请求在执行队列中等了多久。

Batch Wait: 为了凑 Batch 等了多久。

Compute Time: 模型真正计算了多久。

Serialization Time: 输入输出格式转换花了多久。

p50: 普通请求的延迟。

p95: 较慢请求的延迟。

p99: 极端长尾延迟。

TTFT: 多久看到第一个 Token。

TPOT: 之后平均生成一个 Token 要多久。

ITL: 相邻 Token 的实际到达间隔。

tokens/s: 每秒处理或生成多少 Token。